```
# Importing the Dataset

from google.colab import files

print("Please upload the 'Apple Dataset.csv' file.")
uploaded = files.upload()
```

```
Please upload the 'Apple Dataset.csv' file.
```

```
<IPython.core.display.HTML object>
```

```
Saving Apple Dataset.csv to Apple Dataset.csv
```

```
# Importing Libraries

# Data Analysis
import pandas as pd
import numpy as np

# Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

# Loading the Dataset

df = pd.read_csv("Apple Dataset.csv")

# Displaying the Dataset

df.head() # First 5 rows
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 10954,\n  \"fields\":
[\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n
\"dtype\": \"object\",\n        \"num_unique_values\": 10954,\n
\"samples\": [\n          \"2002-05-21\",\n          \"1984-07-05\",\n
\"1990-06-19\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Open\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.458386390498646,\n        \"min\": 0.049665,\n
\"max\": 198.020004,\n        \"num_unique_values\": 6262,\n
\"samples\": [\n          5.903571,\n          15.505357,\n
77.650002\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"High\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.93185612548114,\n        \"min\": 0.049665,\n
```

\"max\": 199.619995,\n   \"num_unique_values\": 6205,\n
\"samples\": [\n   78.167503,\n   0.316607,\n
52.185001\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Low\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 44.013578400204935,\n   \"min\": 0.049107,\n
\"max\": 197.0,\n   \"num_unique_values\": 6194,\n
\"samples\": [\n   29.25,\n   18.178572,\n
48.287498\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Close\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 44.49248300763094,\n   \"min\": 0.049107,\n
\"max\": 198.110001,\n   \"num_unique_values\": 6370,\n
\"samples\": [\n   146.100006,\n   0.229911,\n
6.63\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Adj
Close\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 44.038943348153076,\n   \"min\": 0.0379,\n
\"max\": 197.589523,\n   \"num_unique_values\": 7938,\n
\"samples\": [\n   168.890915,\n   19.281755,\n
172.756729\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Volume\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 335744568,\n   \"min\": 0,\n   \"max\":
7421640800,\n   \"num_unique_values\": 10390,\n
\"samples\": [\n   42291200,\n   94785600,\n
260265600\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"df"}

```
df.tail() # Last 5 rows
```

{"repr_error":"0","type":"dataframe"}

```
# Dataset Information

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10954 entries, 0 to 10953
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Date       10954 non-null  object 
 1   Open       10954 non-null  float64
 2   High       10954 non-null  float64
 3   Low        10954 non-null  float64
 4   Close      10954 non-null  float64
 5   Adj Close  10954 non-null  float64
 6   Volume     10954 non-null  int64  
```

```
dtypes: float64(5), int64(1), object(1)
memory usage: 599.2+ KB
```

```
# Dataset Names
```

```
df.shape
```

```
(10954, 7)
```

```
# Column Names
```

```
df.columns
```

```
Index(['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'],
dtype='object')
```

```
# Cheking missing Values
```

```
df.isnull().sum()
```

```
Date         0
Open         0
High         0
Low          0
Close        0
Adj Close    0
Volume       0
dtype: int64
```

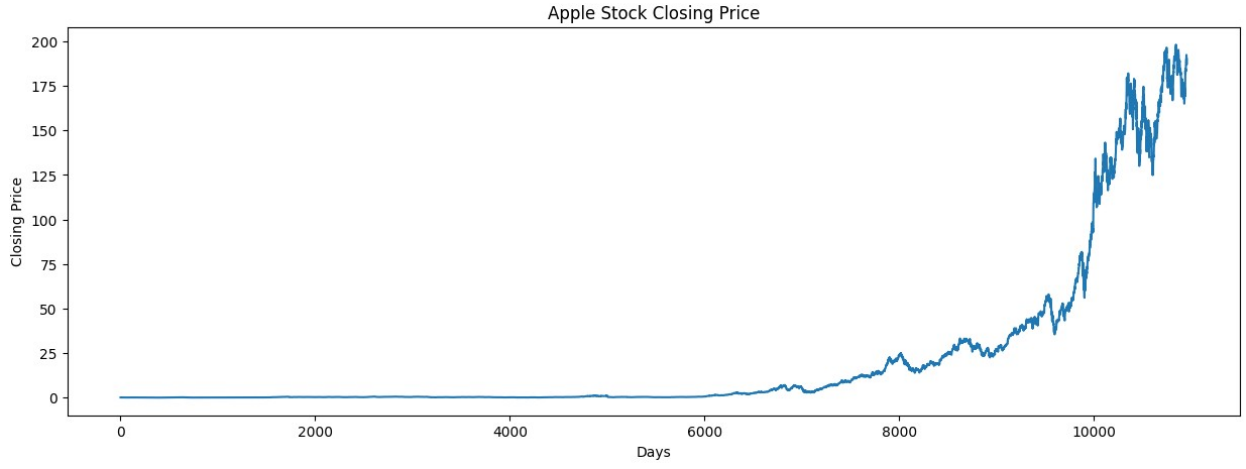
```
# Statical Summary
```

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3859.011063517763,\n
\"min\": 0.049665,\n        \"max\": 10954.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
21.530877479185683,\n          0.522321,\n          10954.0\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"High\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3858.896810588086,\n        \"min\": 0.049665,\n
\"max\": 10954.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          21.761904239364615,\n          0.533482,\n
10954.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3859.1071721616363,\n        \"min\": 0.049107,\n
\"max\": 10954.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          21.308220013511047,\n          0.513393,\n
```

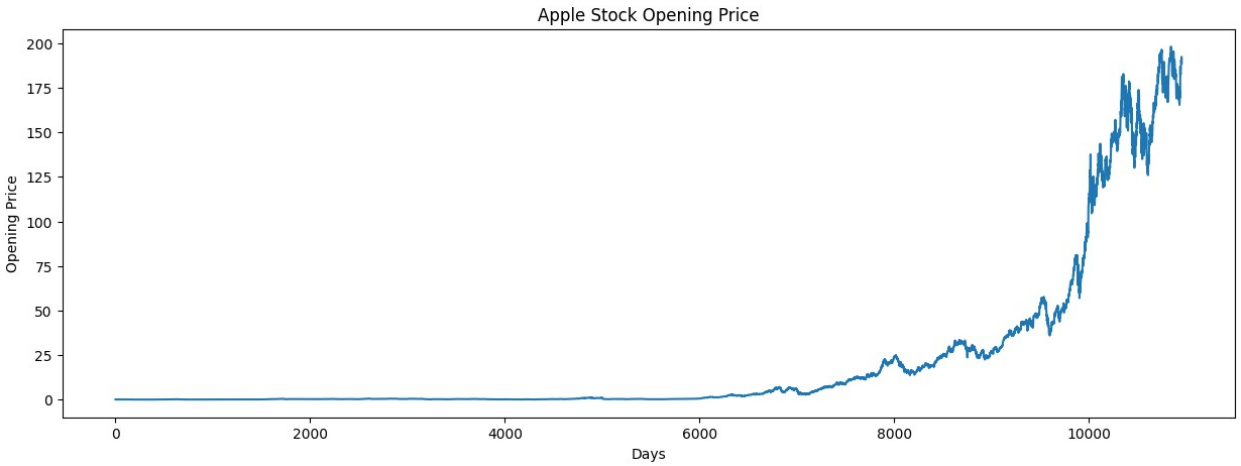
```
10954.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3859.008912463573,\n        \"min\": 0.049107,\n
\"max\": 10954.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          21.544071848365896,\n          0.524554,\n
10954.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Adj
Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 3859.2394938791463,\n        \"min\": 0.0379,\n
\"max\": 10954.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          20.747506455267484,\n
0.42733299999999996,\n          10954.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Volume\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
2558965456.0836005,\n        \"min\": 0.0,\n        \"max\":
7421640800.0,\n        \"num_unique_values\": 8,\n        \"samples\":
[\n          319079187.9222202,\n          206712800.0,\n
10954.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```

```
# Closing Price Trend

plt.figure(figsize=(15,5))
plt.plot(df['Close'])
plt.title("Apple Stock Closing Price")
plt.xlabel("Days")
plt.ylabel("Closing Price")
plt.show()
```

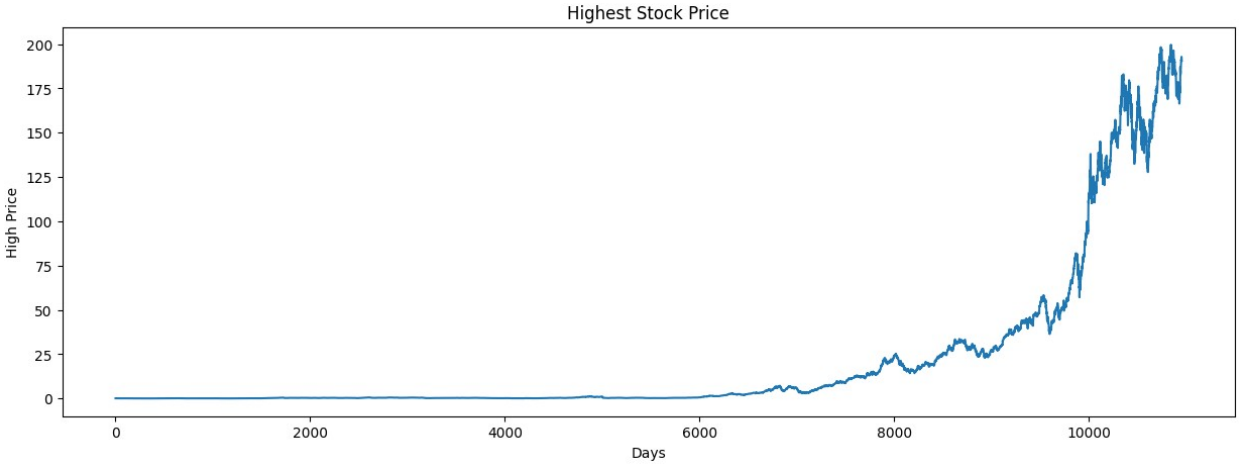



```
# Opening Price Trend

plt.figure(figsize=(15,5))
plt.plot(df['Open'])
```

```
plt.title("Apple Stock Opening Price")
plt.xlabel("Days")
plt.ylabel("Opening Price")
plt.show()
```

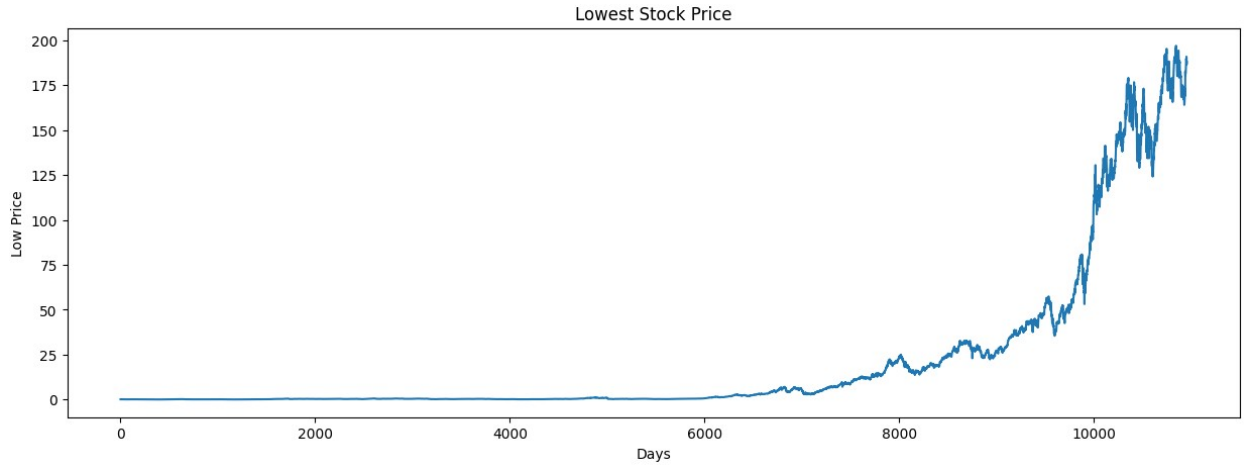



```
# High price trend

plt.figure(figsize=(15,5))
plt.plot(df['High'])
plt.title("Highest Stock Price")
plt.xlabel("Days")
plt.ylabel("High Price")
plt.show()
```

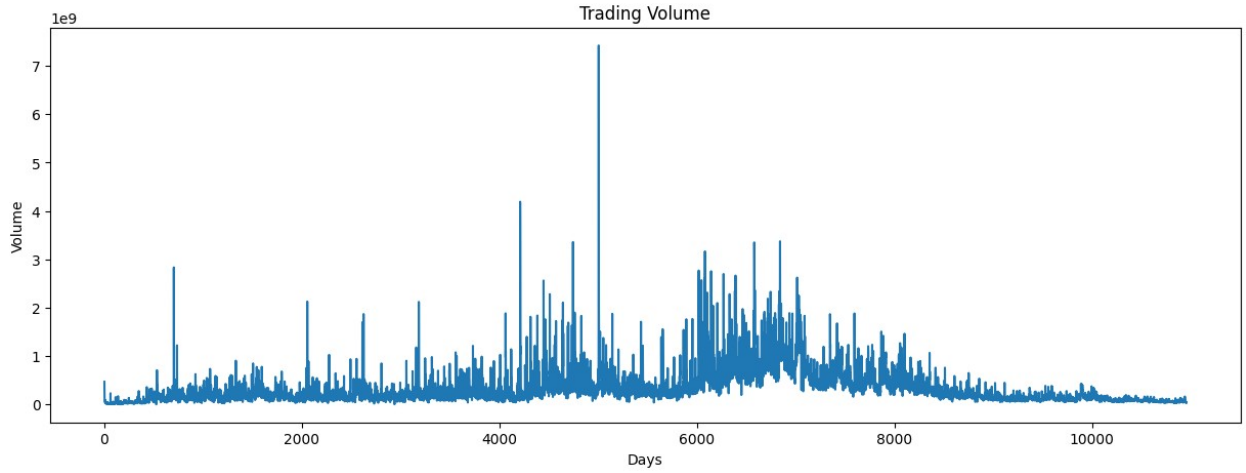



```
# Low Price Trend

plt.figure(figsize=(15,5))
plt.plot(df['Low'])
plt.title("Lowest Stock Price")
```

```
plt.xlabel("Days")
plt.ylabel("Low Price")
plt.show()
```



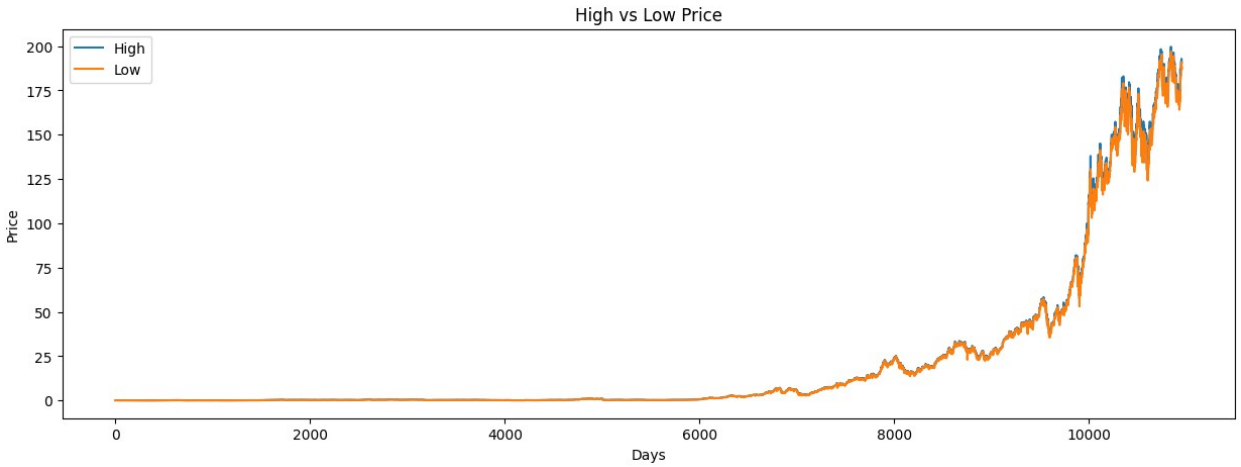

```
# Trading Volume

plt.figure(figsize=(15,5))
plt.plot(df['Volume'])
plt.title("Trading Volume")
plt.xlabel("Days")
plt.ylabel("Volume")
plt.show()
```



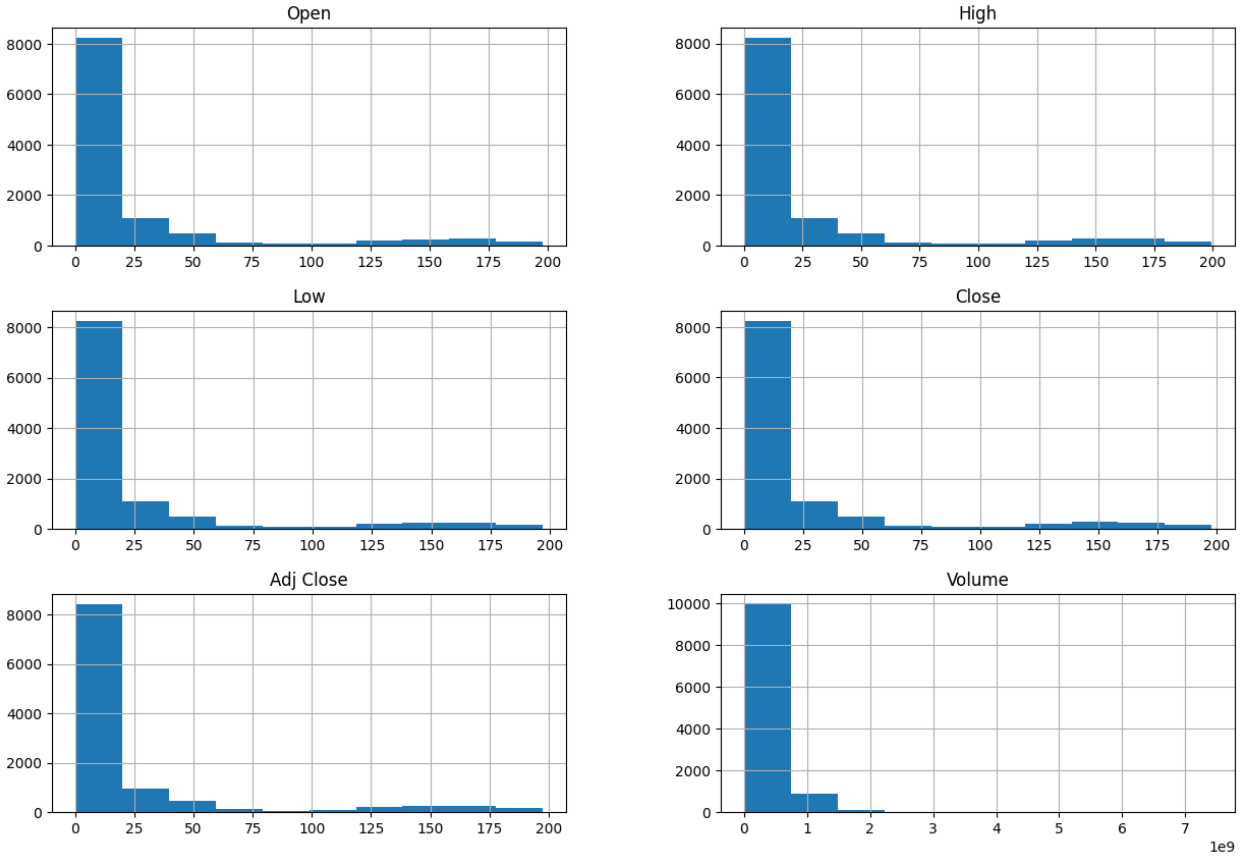

```
# Comparing Opening and Closing Price

plt.figure(figsize=(15,5))

plt.plot(df['Open'], label='Open')
plt.plot(df['Close'], label='Close')
```

```
plt.title("Open vs Close Price")
plt.xlabel("Days")
plt.ylabel("Price")
plt.legend()

plt.show()
```


```
# Comparing High and Low Prices

plt.figure(figsize=(15,5))

plt.plot(df['High'], label='High')
plt.plot(df['Low'], label='Low')

plt.title("High vs Low Price")
plt.xlabel("Days")
plt.ylabel("Price")
plt.legend()

plt.show()
```




```
# Histogram

df.hist(figsize=(15,10))
plt.show()
```

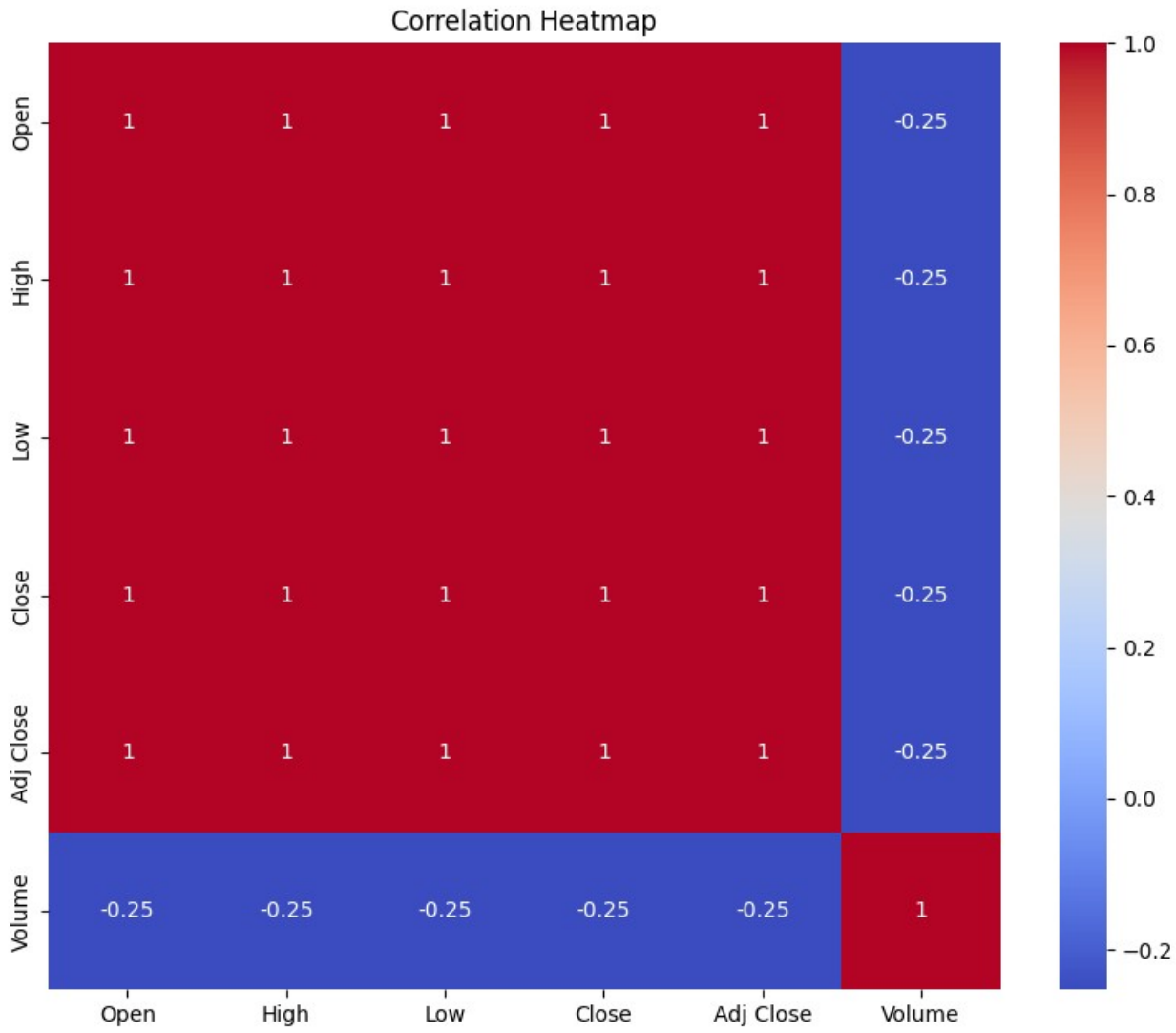



```
# Correlation Matrix

df.corr(numeric_only=True)
```

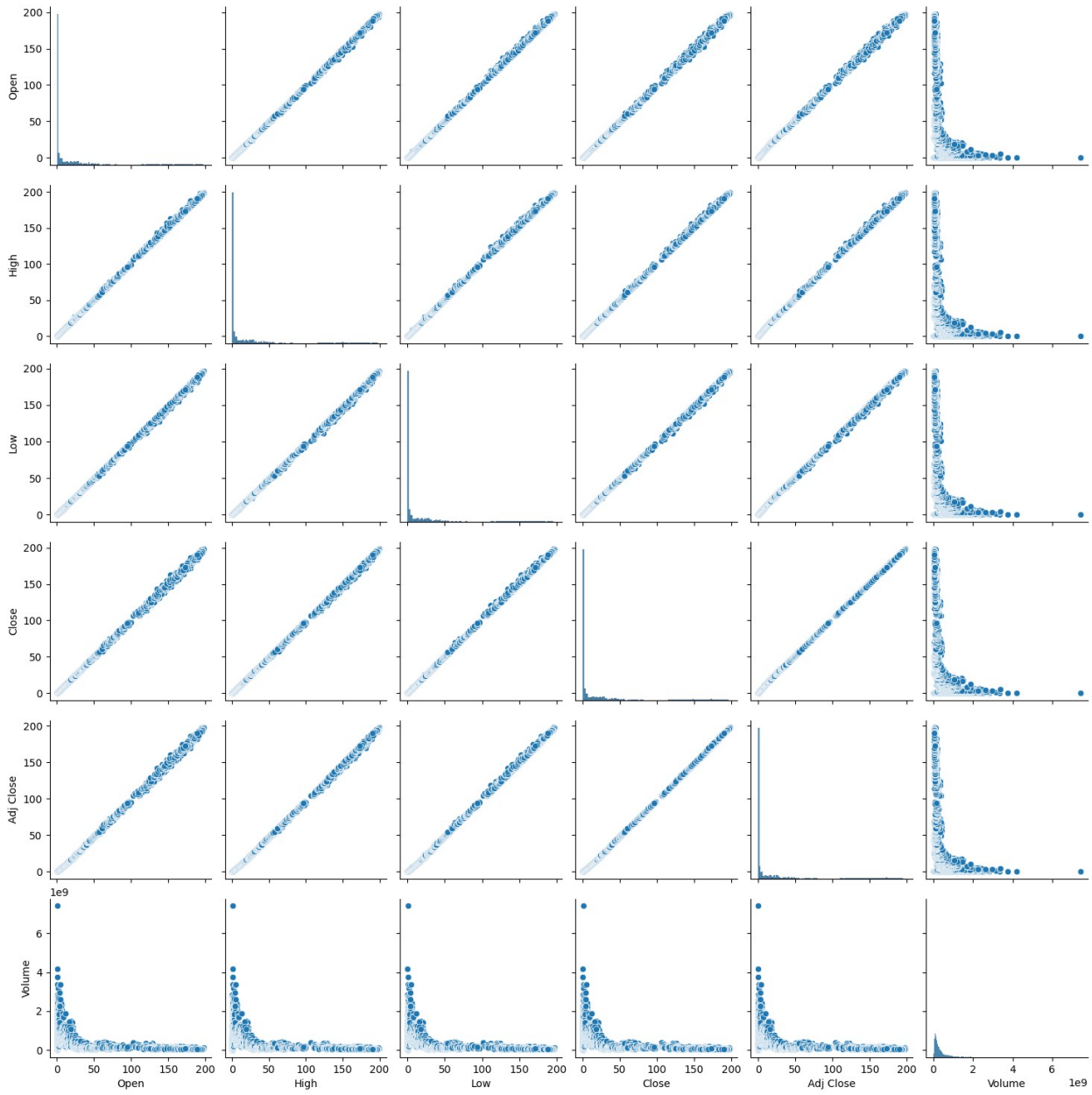
{"summary":"{\n \"name\": \"df\",\n \"rows\": 6,\n \"fields\": [\n{\n   \"column\": \"Open\",\n   \"properties\": {\n\"dtype\": \"number\",\n   \"std\": 0.5108379945076833,\n\"min\": -0.251407290224022,\n   \"max\": 1.0,\n\"num_unique_values\": 6,\n   \"samples\": [\n   1.0,\n0.9999488276712253,\n   -0.251407290224022\n   ],\n\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"High\",\n   \"properties\": {\n\"dtype\": \"number\",\n   \"std\": 0.510698397867367,\n\"min\": -0.25104276799099134,\n   \"max\": 1.0,\n\"num_unique_values\": 6,\n   \"samples\": [\n0.9999488276712253,\n   1.0,\n   -0.25104276799099134\n],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n}\n   },\n   {\n   \"column\": \"Low\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.5110968916918287,\n   \"min\": -0.25201768638321664,\n   \"max\": 1.0,\n\"num_unique_values\": 6,\n   \"samples\": [\n0.9999406121751571,\n   0.999925636056552,\n   -0.25201768638321664\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Close\",\n   \"properties\": {\n   \"dtype\": \"number\",\n\"std\": 0.5109041974229835,\n   \"min\": -0.2515452863406102,\n\"max\": 1.0,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   0.9998755806197666,\n   0.9999404193055652,\n-0.2515452863406102\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Adj Close\",\n   \"properties\": {\n   \"dtype\": \"number\",\n\"std\": 0.5115388725060237,\n   \"min\": -0.2532276440784397,\n\"max\": 1.0,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   0.9996604973256081,\n   0.9997236040678292,\n-0.2532276440784397\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Volume\",\n   \"properties\": {\n   \"dtype\": \"number\",\n\"std\": 0.5110654219171503,\n   \"min\": -0.2532276440784397,\n\"max\": 1.0,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   -0.251407290224022,\n   -0.25104276799099134,\n1.0\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
# Correlation Heatmap

plt.figure(figsize=(10,8))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")
```

```
plt.show()
```




```
# Pair Plot

sns.pairplot(df)

plt.show()
```




```
# Checkig the Dataset Again

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 10954,\n  \"fields\":
[\n    {\n      \"column\": \"Date\",\n      \"properties\": {\n
\"dtype\": \"object\",\n        \"num_unique_values\": 10954,\n
\"samples\": [\n          \"2002-05-21\",\n          \"1984-07-05\",\n
\"1990-06-19\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Open\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.458386390498646,\n        \"min\": 0.049665,\n
```

```
\"max\": 198.020004,\n        \"num_unique_values\": 6262,\n
\"samples\": [\n          5.903571,\n          15.505357,\n
77.650002\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"High\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.93185612548114,\n        \"min\": 0.049665,\n
\"max\": 199.619995,\n        \"num_unique_values\": 6205,\n
\"samples\": [\n          78.167503,\n          0.316607,\n
52.185001\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Low\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.013578400204935,\n        \"min\": 0.049107,\n
\"max\": 197.0,\n        \"num_unique_values\": 6194,\n
\"samples\": [\n          29.25,\n          18.178572,\n
48.287498\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.49248300763094,\n        \"min\": 0.049107,\n
\"max\": 198.110001,\n        \"num_unique_values\": 6370,\n
\"samples\": [\n          146.100006,\n          0.229911,\n
6.63\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Adj
Close\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 44.038943348153076,\n        \"min\": 0.0379,\n
\"max\": 197.589523,\n        \"num_unique_values\": 7938,\n
\"samples\": [\n          168.890915,\n          19.281755,\n
172.756729\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Volume\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 335744568,\n        \"min\": 0,\n        \"max\":
7421640800,\n        \"num_unique_values\": 10390,\n
\"samples\": [\n          42291200,\n          94785600,\n
260265600\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


```
# Checking Data Types

df.dtypes
```

```
Date          object
Open         float64
High         float64
Low          float64
Close        float64
Adj Close    float64
Volume         int64
dtype: object
```

```
# Converting Date Column

df['Date'] = pd.to_datetime(df['Date'])

# Again cheking

df.dtypes
```

```
Date          datetime64[ns]
Open                 float64
High                 float64
Low                  float64
Close                float64
Adj Close            float64
Volume                 int64
dtype: object
```

```
# Selecting the Features (X)

X = df[['Open', 'High', 'Low', 'Volume']]

# Display

X.head()
```


```
{"summary":"{\n  \"name\": \"X\",\n  \"rows\": 10954,\n  \"fields\":
[\n    {\n      \"column\": \"Open\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 44.458386390498646,\n
\"min\": 0.049665,\n        \"max\": 198.020004,\n
\"num_unique_values\": 6262,\n        \"samples\": [\n
5.903571,\n          15.505357,\n          77.650002\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"High\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 44.93185612548114,\n
\"min\": 0.049665,\n        \"max\": 199.619995,\n
\"num_unique_values\": 6205,\n        \"samples\": [\n
78.167503,\n          0.316607,\n          52.185001\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Low\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 44.013578400204935,\n
\"min\": 0.049107,\n        \"max\": 197.0,\n
\"num_unique_values\": 6194,\n        \"samples\": [\n
29.25,\n          18.178572,\n          48.287498\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Volume\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 335744568,\n
\"min\": 0,\n        \"max\": 7421640800,\n
\"num_unique_values\": 10390,\n        \"samples\": [\n
42291200,\n          94785600,\n          260265600\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"X"}
```

```
# Selecting the Target (y)

y = df['Close']

# Display

y.head()
```

```
0    0.128348
1    0.121652
2    0.112723
3    0.115513
4    0.118862
Name: Close, dtype: float64
```

```
# Cheking Shapes

print(X.shape)
print(y.shape)
```

```
(10954, 4)
(10954,)
```

```
# Spling the Dataset We are using 80% for training and 20% for Testing

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42)

# Verifying the Split

print(X_train.shape)
print(X_test.shape)

print(y_train.shape)
print(y_test.shape)
```

```
(8763, 4)
(2191, 4)
(8763,)
(2191,)
```

```
# Cheking missing Values Again

X_train.isnull().sum()
```

```
Open      0
High      0
```

```
Low       0
Volume    0
dtype: int64
```

```
X_test.isnull().sum()
```

```
Open      0
High      0
Low       0
Volume    0
dtype: int64
```

```
# Feature Scaling

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# Importing Linear Regression

from sklearn.linear_model import LinearRegression

# Creating the Model

model = LinearRegression()

# Training the Model

model.fit(X_train, y_train)
```

```
LinearRegression()
```

```
# Making Predictions

y_pred = model.predict(X_test)

# Displaying the Preditions

print(y_pred)
```

```
[0.42583327 0.10998027 0.34918354 ... 0.22730115 0.46468642
5.65923297]
```

```
# Comparing Actual vs Predicted

comparison = pd.DataFrame({
    "Actual Price": y_test,
    "Predicted Price": y_pred
})
```

```
comparison.head(10)
```


```
{"summary":"{\n  \"name\": \"comparison\",\n  \"rows\": 2191,\n
\"fields\": [\n    {\n      \"column\": \"Actual Price\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
44.639697032407696,\n        \"min\": 0.054688,\n        \"max\":
195.830002,\n        \"num_unique_values\": 1646,\n
\"samples\": [\n          23.68,\n          160.550003,\n
0.383929\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Predicted Price\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 44.67147902239595,\n        \"min\":
0.05430022083658759,\n        \"max\": 197.0504101994722,\n
\"num_unique_values\": 2191,\n        \"samples\": [\n
25.017412876612354,\n          0.868806427121358,\n
1.2813356332863333\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"comparison"}
```


```
# Calculating the MAE

from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)

print("Mean Absolute Error:", mae)
```

```
Mean Absolute Error: 0.09949916742984029
```

```
# Calculating the MSE

from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)

print("Mean Squared Error:", mse)
```

```
Mean Squared Error: 0.08774116411213148
```

```
# Calculating the RMSE

import numpy as np

rmse = np.sqrt(mse)

print("Root Mean Squared Error:", rmse)
```

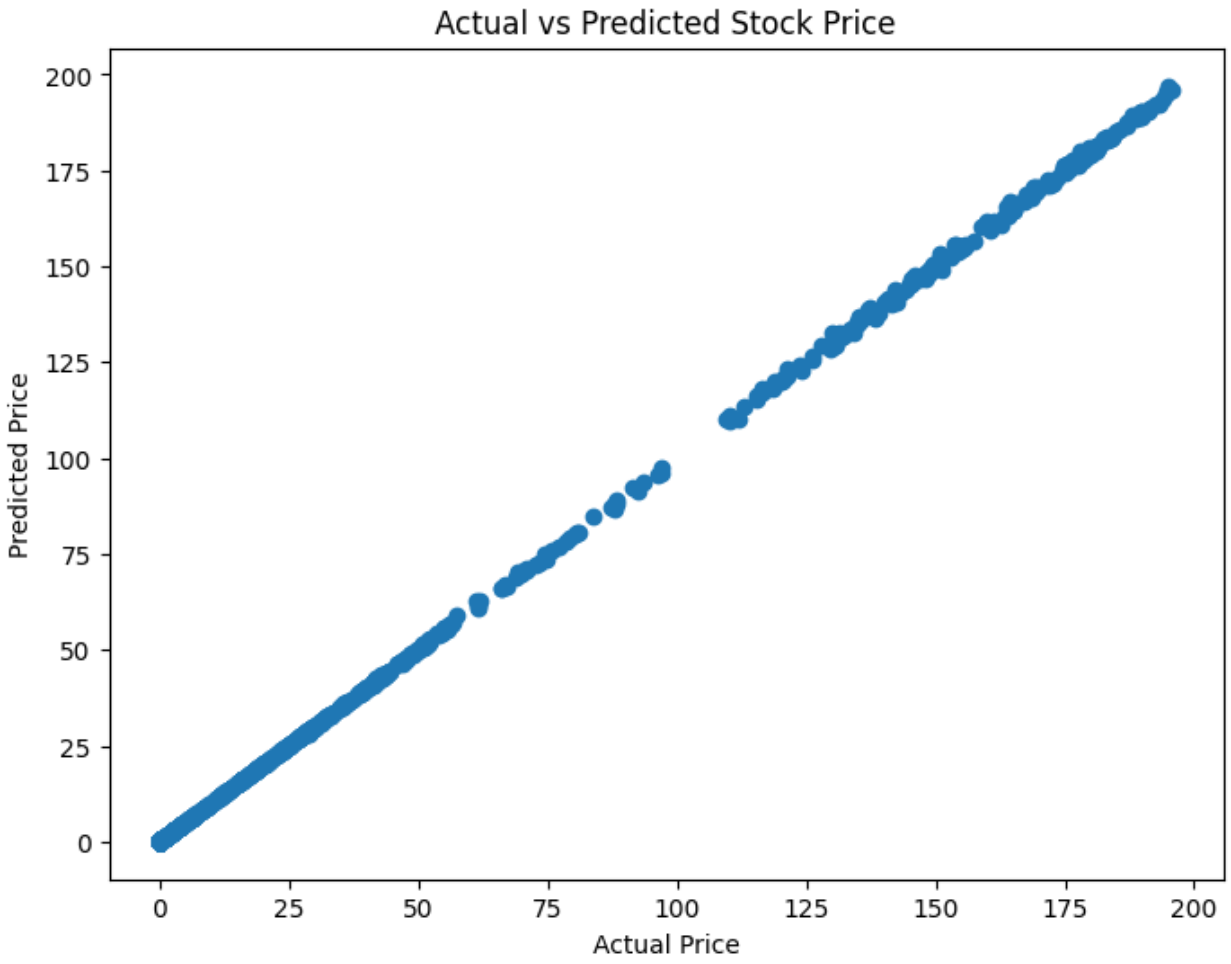
```
Root Mean Squared Error: 0.2962113504107016
```

```
# Calculating R^2 Score
```

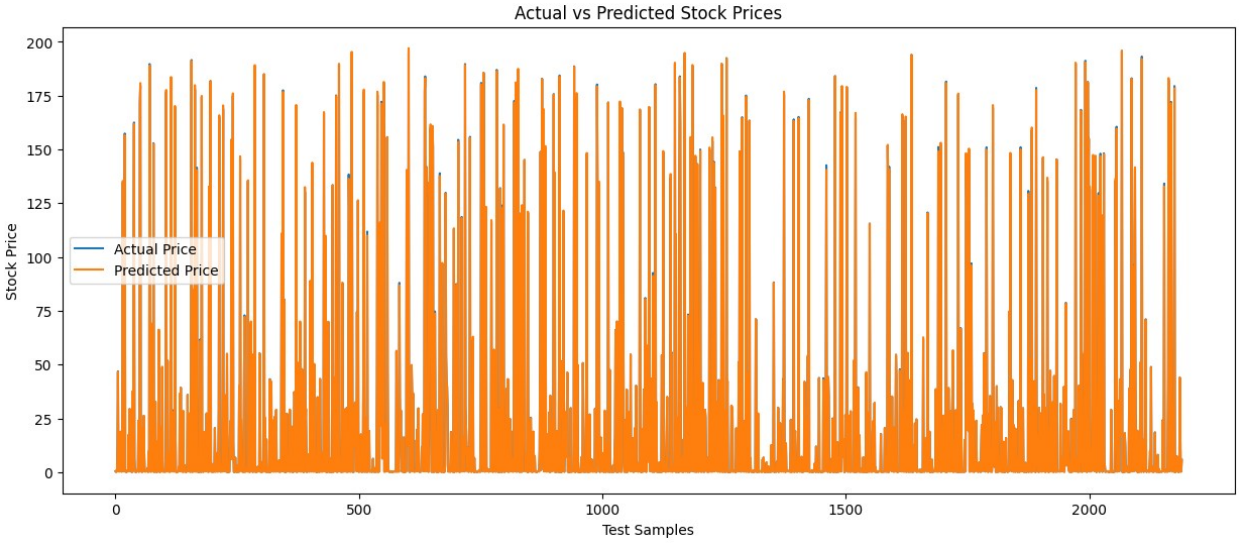
```python
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)

print("R2 Score:", r2)
```

```
R2 Score: 0.9999559486544943
```

```python
# Scatter Plot (Actual vs Predicted)

plt.figure(figsize=(8,6))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")

plt.title("Actual vs Predicted Stock Price")

plt.show()
```



```
# Line Graph Comparison

comparison = comparison.reset_index(drop=True)

plt.figure(figsize=(15,6))

plt.plot(comparison['Actual Price'], label='Actual Price')
plt.plot(comparison['Predicted Price'], label='Predicted Price')

plt.title("Actual vs Predicted Stock Prices")
plt.xlabel("Test Samples")
plt.ylabel("Stock Price")

plt.legend()

plt.show()
```




```
# Displaying Model Coefficients

print("Intercept:", model.intercept_)

print("Coefficients:", model.coef_)
```

```
Intercept: -0.0010611159112414725
Coefficients: [-6.49388344e-01  8.38269261e-01  8.11200564e-01
5.81901194e-12]
```

```
# Pedicting a New Sock Price

new_data = [[180.50, 183.20, 179.90, 55000000]]

prediction = model.predict(new_data)
```

```
print("Predicted Closing Price:", prediction[0])
```

```
Predicted Closing Price: 182.2905728448751
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LinearRegression was fitted with feature names
  warnings.warn(
```